Università degli Studi di Modena e Reggio Emilia

Facoltà di Ingegneria

CORSO DI

RETI DI CALCOLATORI

TCP e UDP:
Il Livello di Trasporto Internet

Prof. Franco Zambonelli

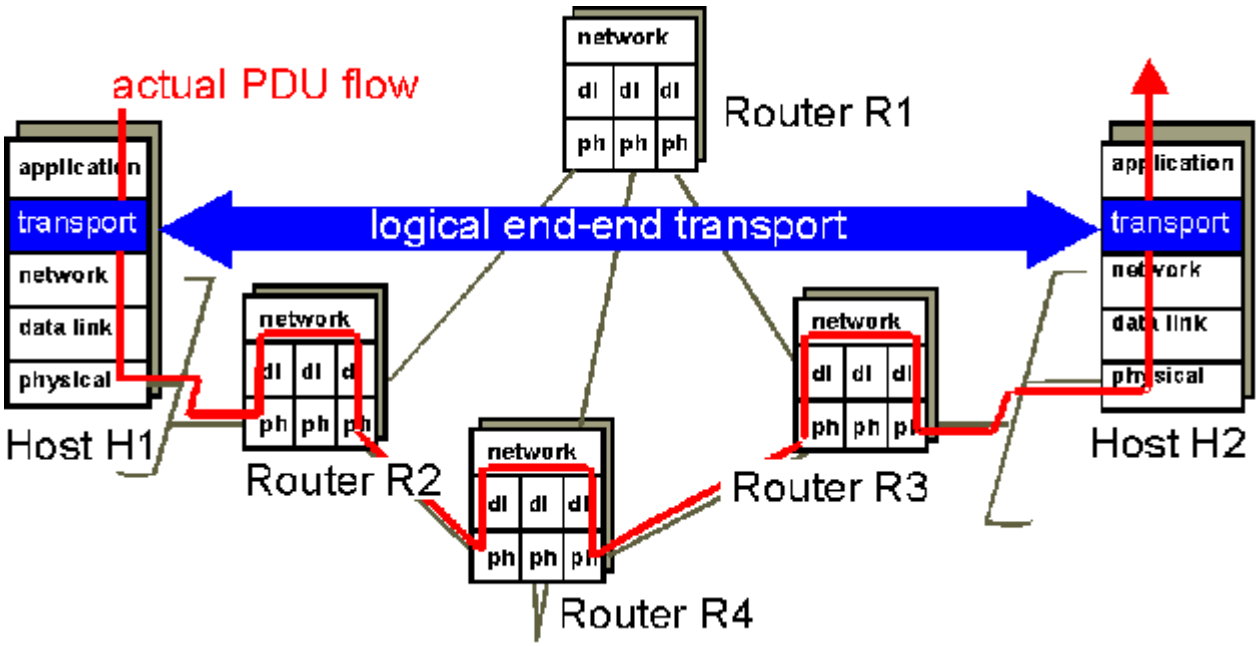
(lucidi realizzati con il contributo di Valeria Cardellini e Michele Colajanni)

Anno Accademico 2001-2002

IL LIVELLO DI TRASPORTO (1)

E' situato tra strato di rete e strato di applicazione

– fornisce un servizio di ***comunicazione logica*** tra processi di livello applicativo in esecuzione su computer ("**host**") diversi
  - *comunicazione logica*: i processi applicativi comunicanti non sono connessi fisicamente tra di loro
  - ma attraverso lo strato di trasporto è come se lo fossero



Importante il concetto di **PROCESSO** (o THREAD) su un sistema operativo concorrente o multiprogrammato!

- **istanza** di un programma in esecuzione
- più processi che eseguono programmi diversi (p.e., Explorer, Napster)
- più processi che eseguono lo stesso programma (p.e., due finestre separate di Explorer)

Le comunicazioni non dovranno più essere solo tra computer e computer, ma tra processi e processi su un altro computer: ci pensa il livello di trasporto Internet!

Il LIVELLO DI TRASPORTO (2)

Il protocollo di trasporto è implementato soltanto nei nodi terminali, non nei router di rete

Il protocollo di rete agisce sui protocolli di livello 3, il protocollo di trasporto sui protocolli di livello 4;

Il protocollo di trasporto:

dal lato mittente, converte il messaggio ricevuto dal livello applicativo (p.e., la richiesta di spedire un file HTML), eventualmente decomponendolo in parti più piccole, attraverso i protocolli di livello 4, e aggiungendo l'header di livello 4;

passa il messaggio allo strato di rete, che lo incapsula con il suo header di livello 3 e lo instrada…

dal lato destinatario, si ricevono messaggi (IP datagram) dal livello 3, si elabora il messaggio attraverso l'esecuzione dei necessari protocolli di livello 4 (per esempio per riassemblare il messaggio se era stato spezzato), si toglie l'header di livello 4, e lo passa al processo applicativo;

NOTA (RICORDIAMO CHE):
ogni livello aggiunge il suo header man mano che i dati da trasmettere passano dall'alto al basso nella pila dei livelli.
Viceversa, per i dati ricevuti che passano dal basso all'alto nella pila, ogni livello toglie il suo header prima di passare il messaggio in alto nella pila

Può esserci più di un protocollo di trasporto. In Internet:

User Datagram Protocol (UDP)

Transmission Control Protocol (TCP)

Servizi dello strato di trasporto

Fornisce la *comunicazione logica tra processi* in esecuzione su host diversi

Lo strato di rete fornisce solo un servizio di *comunicazione logica tra host*

Esempio: servizio postale tra due case abitate da cugini in cui la corrispondenza avviene tra più cugini:

- host = casa,
- processi = cugini,
- messaggio = lettera nella busta,
- protocollo di livello di rete = servizio postale,
- protocollo di livello di trasporto = cugini addetti in ciascuna casa allo smistamento della posta,

I servizi, in particolare, sono
– servizio di **multiplazione/demultiplazione**, per consegnare messaggi a processi specifici su un host
– **trasferimento affidabile** dei dati, per controllare che i messaggi siano arrivati
– garanzia della **larghezza di banda**, per evitare di spedire più messaggi di quanti ne possano essere ricevuti (parlare troppo in fretta…)
– garanzia del **ritardo**, per evitare tempi di attesa illimitati nella consegna dei messaggi (la rete Internet è condivisa e i suoi intasamenti non controllabili)

UDP e TCP

Ricordiamo le Caratteristiche principali del protocollo di rete IP:
– comunicazione logica tra host
– servizio di consegna con impegno (si impegna, ma non garantisce la consegna del messaggio)
– servizio non affidabile

In Internet sono disponibili due distinti protocolli di trasporto:
– ***User Datagram Protocol*** (UDP)
– ***Transmission Control Protocol*** (TCP)
Hanno caratteristiche che estendono le funzionalità del livello 3, alcune comuni a entrambi, altre solo del TCP

Servizi comuni a UDP e TCP:
– estensione del servizio di consegna del protocollo IP tra due nodi a terminali ad un servizio di consegna a due processi applicativi in esecuzione sui nodi terminali
multiplazione e ***demultiplazione***– rilevamento dell'errore

Servizi aggiuntivi di TCP:
– servizio con connessione tra cliente e servitore
– trasferimento affidabile dei dati
controllo di flusso, numeri di sequenza, acknowledgement e timer
– controllo di congestione
regola il tasso di invio dei messaggi da parte del mittente

NOTA: a livello 4 spesso si parla di segmenti più che di messaggi! Quindi: messaggi = livelli 5-7, segmenti livello 4, datagram livello 3, frame o pacchetti a livello 2

MULTIPLAZIONE E DEMULTIPLAZIONE

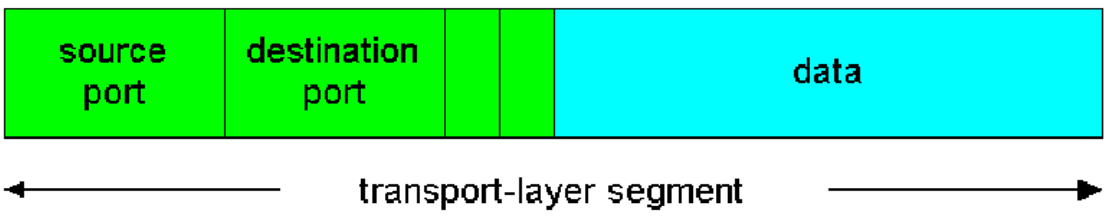
Meglio: multiplexing e demultiplexing

–Il protocollo IP non consegna i dati tra processi applicativi in esecuzione sui nodi terminali → lo fa il protocollo di trasporto
un indirizzo IP per identificare ogni nodo terminale

– Ogni segmento dello strato di trasporto possiede un campo contenente l'informazione usata per determinare a quale processo deve essere consegnato il segmento → **demultiplazione**
la demultiplazione avviene dal lato del nodo destinatario

– Creazione dei segmenti provenienti dai messaggi di diversi processi applicativi → **multiplazione**
la multiplazione avviene dal lato del nodo mittente

UDP e TCP attuano la multiplazione/demultiplazione includendo due campi speciali nell'header del segmento:
– il ***numero di porta del mittente***
– il ***numero di porta del destinatario***
permettono di identificare in modo univoco i due processi applicativi, residenti su due nodi terminali e comunicanti tra loro.



NUMERI DI PORTA

Le porte (o “socket”, come le prese elettriche) sono astrazioni per il software di rete

Rappresentano dei punti di connessioni virtuali di un processo verso la rete a livello 4

Le porte su un computer si identificano a livello di processo tramite numeri interi

- Numero di porta: numero di 16 bit compreso tra 0 e 65535
- numeri di porta noti (well-known port numbers): tra 0 e 1023
  riservati per protocolli applicativi noti (p.e. HTTP e FTP)
  HTTP: numero di porta 80
  FTP: numero di porta 21

Quando si sviluppa una nuova applicazione è necessario assegnarle un nuovo numero di porta (non quelle note), o più di uno se è il caso

- I processi della applicazione useranno le porte per mandare messaggi sulla rete e per riceverli
- Processi diversi e applicazioni diverse usano porte diverse

Quindi, a livello di trasporto, una coppia mittente-destinatario è individuata dalla coppia (indirizzo IP, numero di porta).

Esempio: (128.2.254.139, 1184) e (128.2.254.139, 53)

GESTIONE NUMERI DI PORTA:
IL MODELLO CLIENT-SERVER

Come vengono assegnati i numeri di porta?

Modello client/server:

Modello fondamentale per la rete Internet e la programmazione di rete

Processi **SERVER**

Esistono alcuni "processi" sempre attivi, detti "**server**", su siti Internet noti, in attesa di fornire specifici servizi di rete:

- Server Web, Server WAP
- Server ICQ, IRQ, Server Napster, etc. etc.

Queste applicazioni server sono collegate alla rete tramite una porta NOTA, che tutti sanno essere associata a quel servizio (p.e., server Web HTTP porta 80)

I numer di porta a questi processi sono **assegnati a priori**, tra quelli noti, perché devono essere processi identificabili dai

Processi **CLIENTI**

Applicazioni che richiedono la esecuzione dei servizi dei server.

Per richiedere i servizi non hanno bisogno di un numero di porta noto, ne va bene uno qualsiasi:

- i clienti devono sapere a chi chiedere i servizi a priori
- i server non devono sapere prima a chi fornire i servizi

GESTIONE PORTE PER CLIENT-SERVER

Quando il cliente chiede il servizio:

Numero di porta del destinatario nel segmento inviato dal client al server corrisponde al numero di porta del servizio richiesto (ad es. 80 per HTTP)

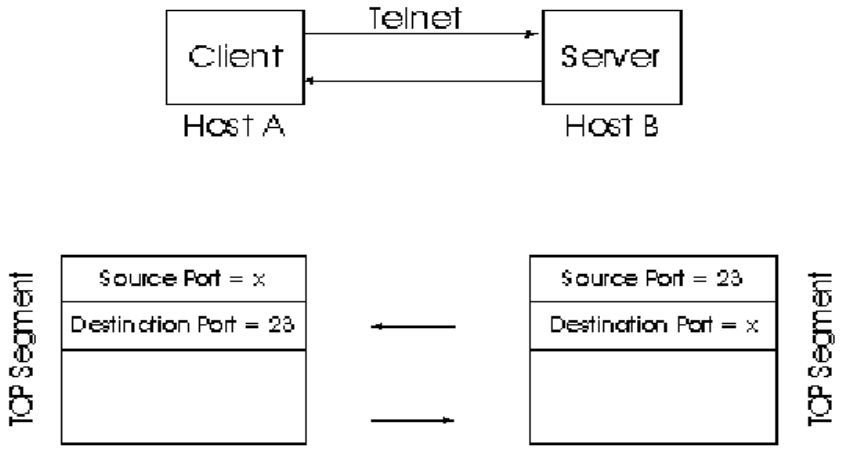
Numero di porta del mittente nel segmento inviato dal client al server corrisponde ad numero di porta scelto tra quelli non già in uso

Quando il server risponde al servizio:

Numero di porta del mittente nel segmento inviato dal server al client corrisponde al numero di porta del servizio richiesto (ad es. 80 per HTTP)

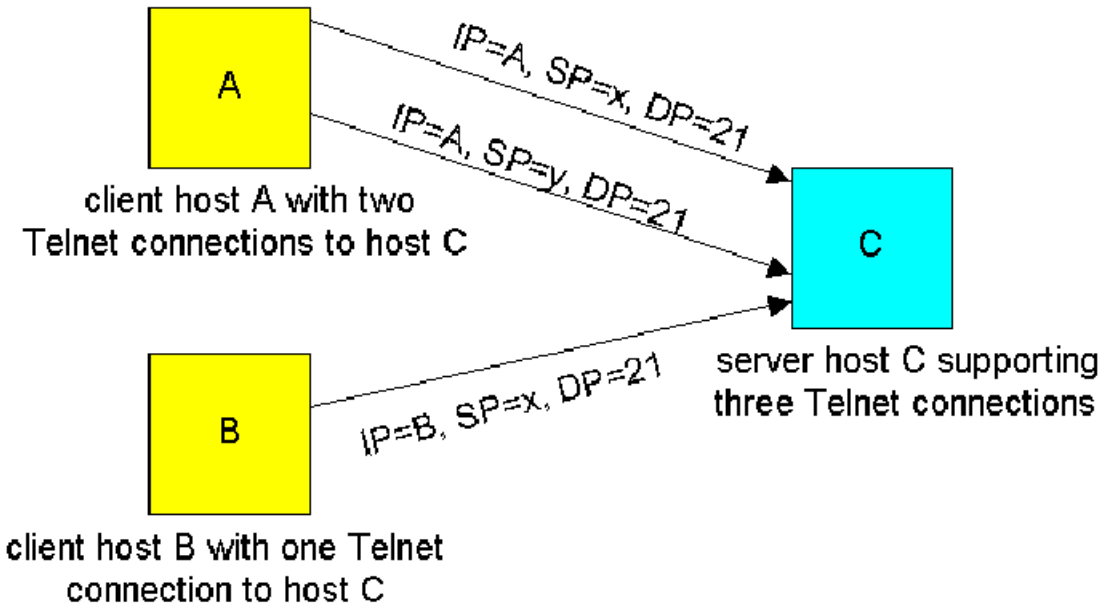
Numero di porta del destinatario nel segmento inviato dal server al client corrisponde ad numero di porta indicato dal client nel messaggio precedentemente inviato

Esempio di uso dei numeri di porta in un'applicazione client/server (es. Telnet, con numero di porta 23):



MULTIPLAZIONE CLIENT-SERVER

Due client, residenti su host diversi e che usano lo stesso numero di porta per comunicare con la stesso servizio applicativo, sono distinti in base al loro indirizzo IP:



NOTA: I server sono in grado di fornire i loro servizi in modo concorrente a più client, attraverso canali di comunicazione porta-porta diversi

OVVIAMENTE: si richiede programmazione concorrente
Ci saranno più processi che eseguono il programma per l'esecuzione del servizio per conto di diversi clienti

INOLTRE: ci sono programmi, composti da più processi, che possono agire sia da client che da server
Esempio: Napster agisce da client quando si prelevano file mp3, ma allo stesso tempo fa da server per chi vuole prelevare file mp3

IL PROTOCOLLO TCP



–Protocollo IP : fornisce un servizio di consegna dei pacchetti privo di connessione (perdita o distruzione dei pacchetti, ritardi, mancanza di ordine nella sequenza)

– I programmi applicativi necessitano di un servizio di trasporto affidabile e con connessioni → **Protocollo TCP (Transmission Control Protocol)**

– UDP non è affidabile e non ha le connessioni

Quasi tutte le applicazioni Internet correnti usano TCP per la caratteristica di affidabilità

Usano UDP soltanto la applicazioni per lo scambio delle NEWS (tanto anche se non arrivano tutte è lo stesso)

Caratteristiche principali TCP

Servizi di multiplazione/demultiplazione, rilevamento dell'errore (no correzione), controllo del flusso e della congestione, e servizio di CONNESSIONE

COSA E' E PERCHE' SERVE LA CONNESSIONE?

TCP è un protocollo orientato alla connessione
Un processo client e un server, quando comunicano, lo fanno attraverso un circuito di comunicazione virtuale che viene instaurato in fase iniziale della comunicazione, e permane per tutta la durata della sessione di comunicazione

La connessione permette a cliente e servitore, prima di iniziare a scambiarsi messaggi, di accordarsi per i meccanismi di controllo del flusso e per la trasmissione affidabile dei messaggi

connessione: fasi di instaurazione, utilizzo e abbattimento della connessione (es. chiamata telefonica)

three-way handshaking: i due processi, prima di iniziare il trasferimento dati, si inviano 3 segmenti per stabilire i parametri della connessione

- connessione punto-punto con unico mittente, unico destinatario
  - connessione full duplex: trasferimento contemporaneo in entrambe le direzioni della connessione

NOTA: è una connessione a livello logico, il livello 2 rimane comunque senza connessione e la rete rimane a commutazione di pacchetto

SERVIZI DI BUFFERIZZAZIONE

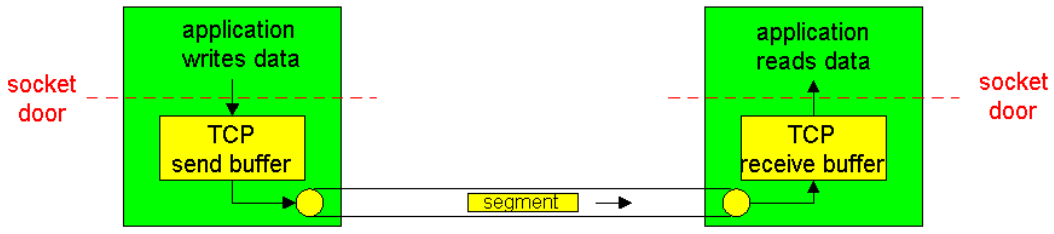
Buffer: area di memoria per la memorizzazione temporanea dei dati prima che questi vengano elaborati

In TCP: area di memoria per la memorizzazione dei segmenti in attesa della trasmissione/ricezione

- trasmissione con buffer: dati ricevuti dal livello applicativo messi temporaneamente in un buffer del mittente prima della elaborazione e della spedizione
- ricezione con buffer: dati ricevuti dalla rete (dal livello 3) messi temporaneamente in un buffer in attesa della elaborazione e del loro passaggio ai livelli superiori

Il livello trasporto non elabora mai direttamente, ma sempre attraverso il buffer

Maximum Segment Size (MSS): dipende dall'implementazione TCP (es. 1500 bye, 536 byte, 512 byte)



La bufferizzazione serve per il controllo del flusso

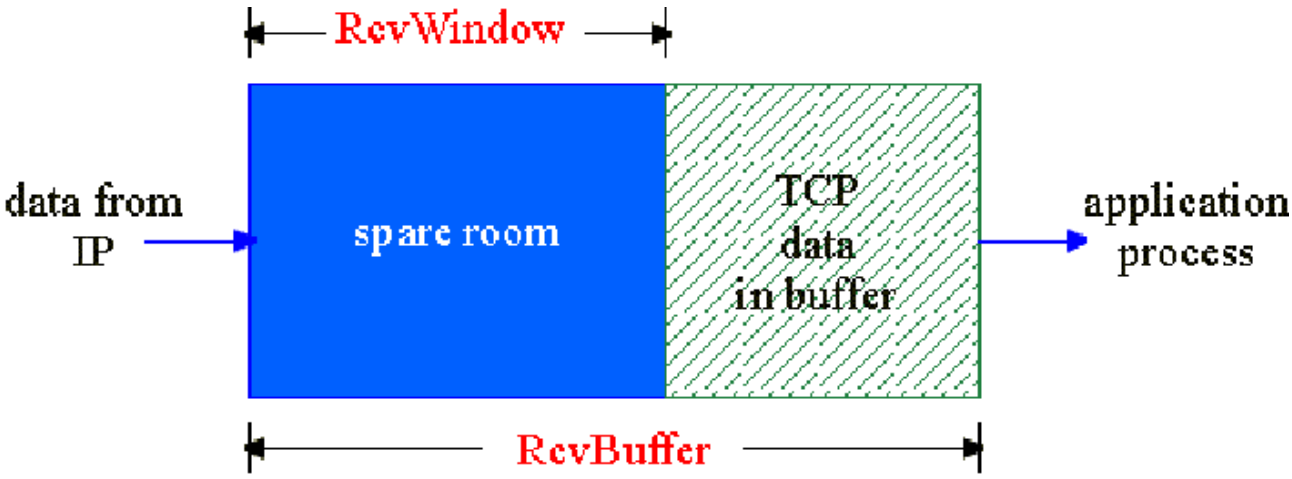
CONTROLLO DEL FLUSSO

–**Controllo di flusso**: il mittente deve evitare di riempie il buffer del destinatario, inviando una quantità eccessiva di dati ad un tasso di trasmissione troppo elevato

Il mittente, grazie al buffer, riesce a ricevere, per tempi limitati, un numero di messaggi superiore a quello che riesce a gestire ed elaborare

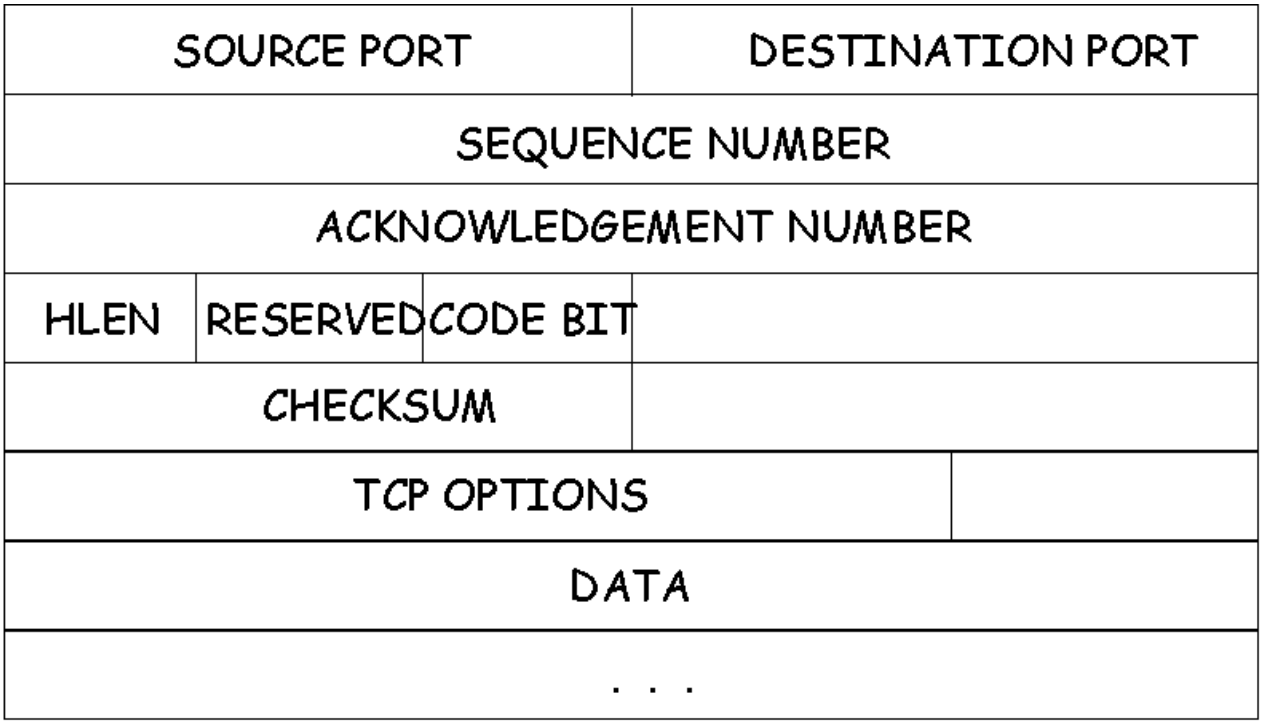
Il destinatario informa esplicitamente il mittente della quantità di spazio libero nel buffer:

dimensione della finestra nel segmento TCP (varia dinamicamente): lo spazio rimasto libero nel buffer



IL SEGMENTO TCP (1)

Formato del segmento:



- source port (16 bit): numero di porta del mittente
- destination port (16 bit): numero di porta del destinatario

- hlen (4 bit): lunghezza dell'header TCP (in multipli di 32 bit)
  se non vi sono opzioni → hlen = 20 byte

- reserved (4 bit): per usi futuri

- window (16 bit): dimensione della finestra
  indica il numero di bit che il destinatario è disposto ad accettare (per il controllo del flusso)

IL SEGMENTO TCP (2)

- code bit (6 bit): scopo e contenuto del segmento
  URG (urgent): dati segnati come urgenti dal livello applicativo sono validi
  ACK (acknowledgement): valore del campo acknowledgement è valido
  PSH (push): il destinatario deve passare i dati all'applicazione immediatamente
  RST (reset), SYN (synchronize), FIN: usati per instaurazione ed abbattimento della connessione

- checksum (16 bit): controllo integrità dei dati trasportati nel segmento

- urgent point (16 bit): puntatore al termine dei dati urgenti

- TCP options: campo opzionale di lunghezza variabile (es. MSS)

- zero padding: per header con lunghezza multipla di 32 bit (se opzioni)

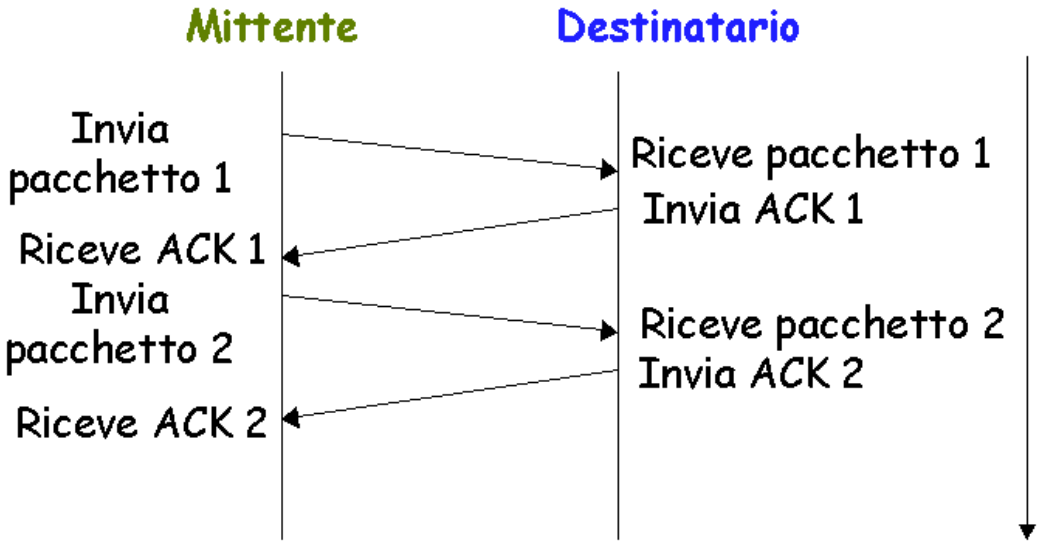
- data: messaggio fornito dal livello applicativo

- sequence number (32 bit): numero di sequenza all'interno della finestra scorrevole di un segmento, per identificare i segmenti

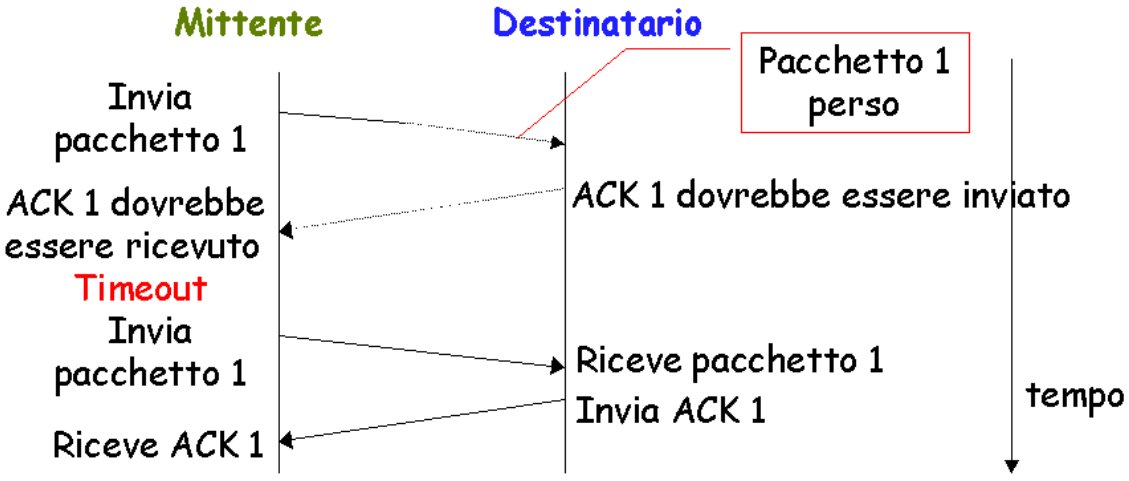
- acknowledgement number (32 bit): ACK relativo ad un numero di sequenza (piggybacking): indica in un segmento il numero di sequenza che ci si aspetta l'ACK di quel messaggio abbia

AFFIDABILITA' in TCP (1)

Uso della tecnica di acknowledgement positivo con ritrasmissione
→ i segmenti inviati sono identificati da un numero di sequenza
→ il destinatario, quando riceve un segmento, invia una conferma, **acknowledgement** (**ACK**), al mittente, per confermare la ricezione di un determinato segmento
→ il mittente attende di ricevere ACK prima di inviare il segmento successivo



Uso della tecnica di acknowledgement positivo con ritrasmissione e timeout → se il mittente non ha ricevuto ACK di un pacchetto dopo un certo periodo (**timeout**), ritrasmette il pacchetto

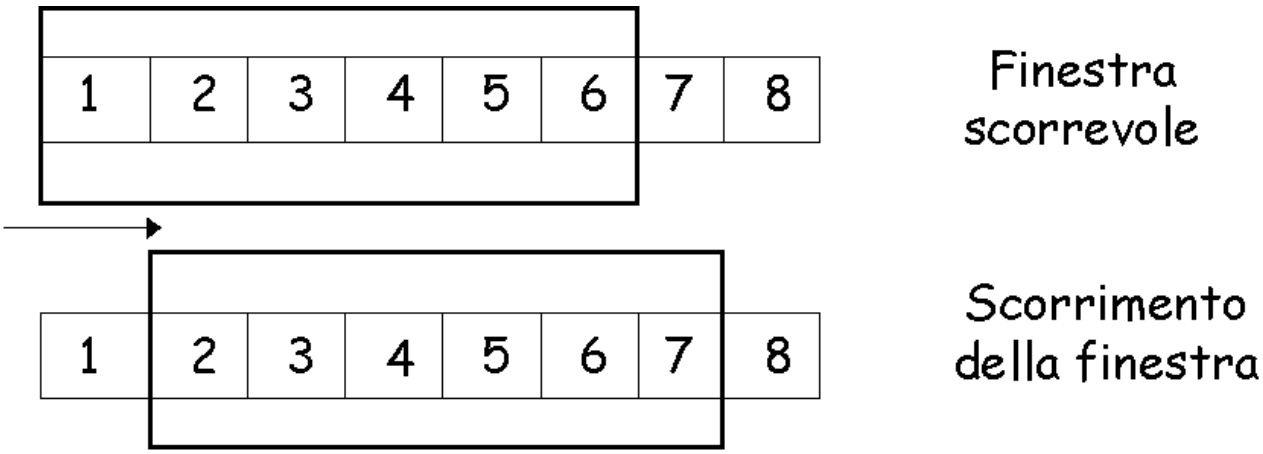


AFFIDABILITA' in TCP (1)

Ack positivo con ritrasmissione e timeout causa perdita di banda della rete (invio di un nuovo pacchetto ritardato fino a che non è stato ricevuto ack del pacchetto precedente) → finestra scorrevole, che scorre via via che viene ricevuto l'ack dei pacchetti già inviati

Esempio: in attesa di ricevere ACK per il segmento 1, si inviano i segmenti 2,3,4, etc….

- Si crea una "finestra" di messaggi in attesa di ACK
- Avanza man mano che vengono ricevuti gli ACK



NOTA: in verità in un sistema distribuito (dove io non vedo cosa succede "altrove") non esistono protocolli di comunicazione affidabili

Problema dei generali bizantini:

- ai due lati di una collina devono decidere se attaccare o no il forte in cima alla collina
- attaccano entrambi o nessuno
- i messaggeri portano le ambascie da un lato all'altro
- da un lato della collina non esiste mai garanzia che i messaggeri siano arrivati dall'altra parte della collina

ROUND TRIP TIME E TIMEOUT (1)

Round Trip Time (Tempo di Andata e Ritorno): Tempo per un invio di un messaggio e ricezione del corrispondente ACK

Come stabilire il valore del timeout del TCP?
– deve essere maggiore del Round Trip Time (RTT)
ma RTT varia dinamicamente

– timeout troppo breve
troppe ritrasmissioni non necessarie

– timeout troppo lungo
reazione lenta alla perdita di segmenti

Come stimare il valore del RTT?

– SampleRTT: misura del tempo trascorso dalla trasmissione del segmento alla ricezione del suo ACK
ignora ritrasmissioni, segmenti con ack cumulativi
SampleRTT varia dinamicamente → usare media pesata

ROUND TRIP TIME E TIMEOUT (2)

–EstimatedRTT: media pesata per stimare RTT

Exponential weighted moving average (EWMA)
influenza dei campioni passati diminuisce in modo esponenziale
valore tipici di x compresi tra 0 e 1

–Timeout: RTT più un margine di errore

Timeout(t) = EstimatedRTT(t) + 4*Deviation(t)

Deviation(t) = (1-x)*Deviation(t-1) +
x*abs(SampleRTT(t)-EstimatedRTT(t))

GESTIONE CONNESSIONE (1)

APERTURA CONNESSIONE

Nel TCP il mittente ed il destinatario, prima di iniziare il trasferimento dei segmenti di dati, instaurano la connessione

– inizializzazione delle variabili del TCP

numeri di sequenza dei segmenti

si crea il buffer, e si scambiano le informazioni per il controllo del flusso (dimensioni buffer e finestra del destinatario)

– modello client/server

client: inizia la connessione

server: contattato dal client

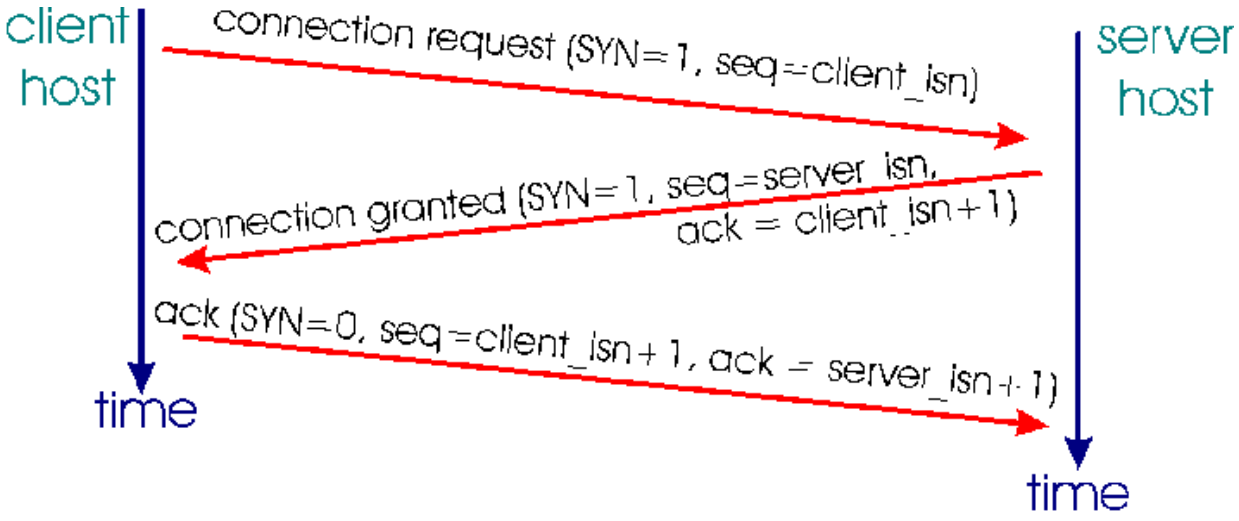
Instaurazione tramite ***Three Way HandShaking***

il client invia un segmento di controllo con SYN=1 al server

il client specifica (nello stesso segmento di controllo) il proprio numero di sequenza per i segmenti, da usare per le conferme

il server riceve il segmento con SYN=1 ed invia un segmento di controllo con SYN=1 e ACK=1

il server specifica (nello stesso segmento di controllo) il proprio numero di sequenza per i segmenti, da usare per le conferme



GESTIONE CONNESSIONE (2)

CHIUSURA DELLA CONNESSIONE

il client invia un segmento di controllo con FIN=1 al server

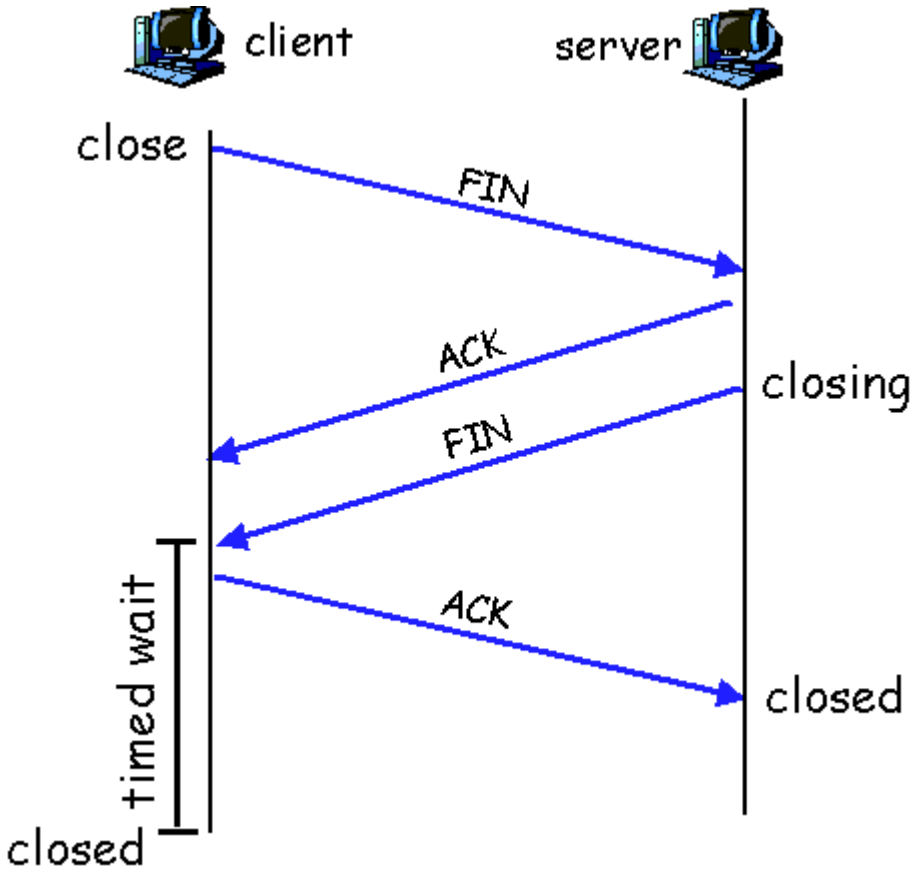
il server riceve FIN, invia ACK

il server chiude la connessione lato client-server ed invia FIN=1 al client

il client riceve il segmento con FIN=1 ed invia ACK

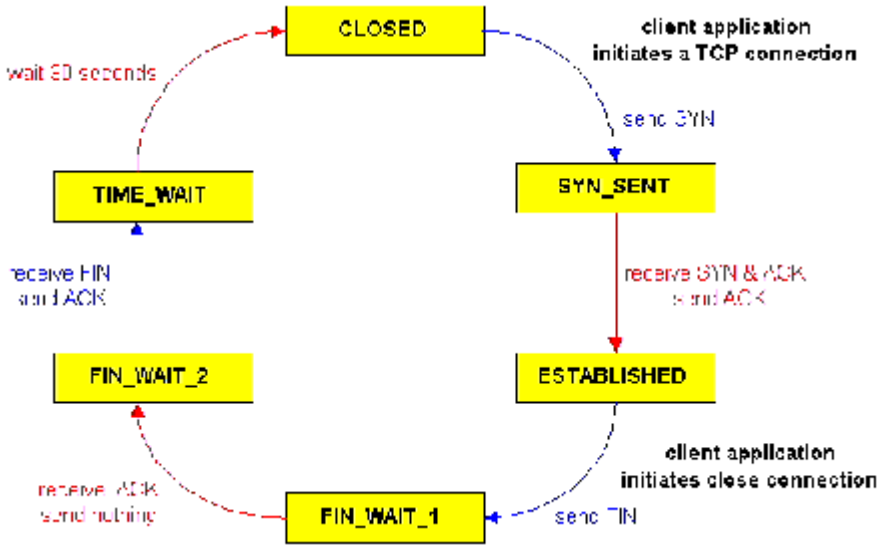
il server riceve ACK

il client attende il timeout dell'ACK inviato; allo scadere anche la connessione lato server-client viene chiusa

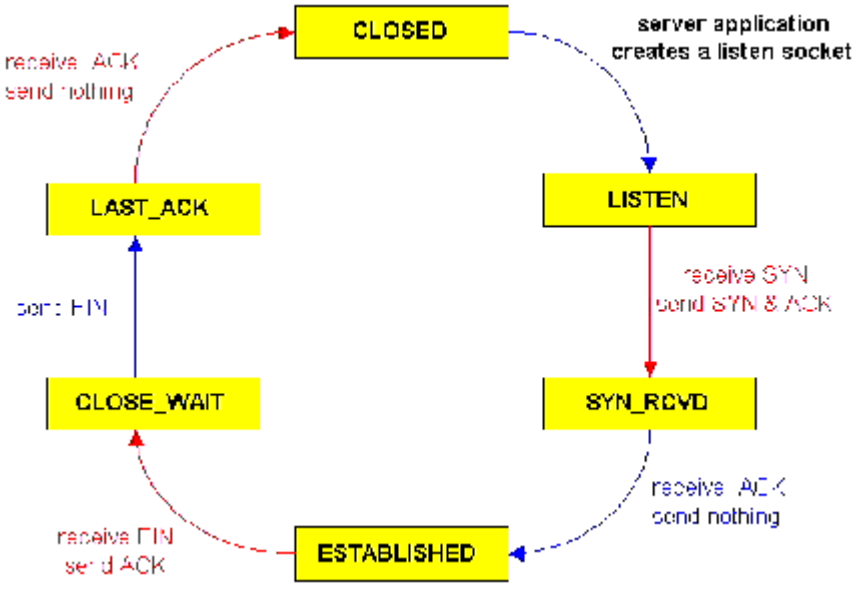


CICLO DI VITA CONNESSIONE

PER CLIENTE



PER SERVITORE



INTERRUZIONE CONNESSIONE

–Reset della connessione

in condizioni normali, la connessione viene abbattuta tramite lo scambio di segmenti di controllo FIN e ACK

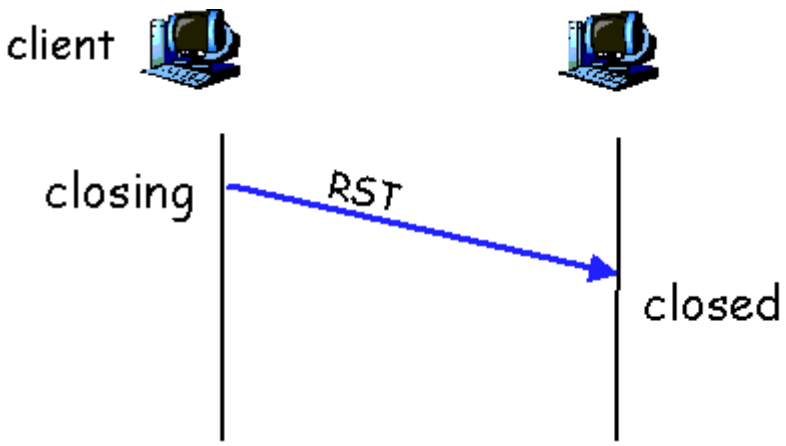
talvolta condizioni anormali che forzano l'applicazione ad interrompere la connessione

TCP fornisce un meccanismo per il reset

il nodo che decide il reset pone RST=1

l'altro nodo chiude immediatamente la connessione

le risorse utilizzate dalla connessione sono rilasciate



CONTROLLO DELLA CONGESTIONE

Congestione:
un numero elevato di sorgenti inviano contemporaneamente troppi dati generando un traffico che la rete (i suoi componenti) non è in grado di sopportare"
In soldini, c'è troppo traffico, si perdono i segmenti, gli ack arrivano in ritardo, si crea ulteriore traffico per la trasmissione, etc. etc.

–**EFFETTI** della congestione:
perdita di pacchetti (overflow dei buffer nei router)
ritardi lunghi (tempi di attesa nei buffer dei router)

–controllo congestione **DIVERSA** da controllo del flusso!

Approcci per il controllo della congestione:

Controllo di congestione end-to-end
lo strato di rete non fornisce supporto allo strato di trasporto
la situazione di congestione è determinata analizzando le perdite di pacchetti ed i ritardi nei nodi terminali
approccio utilizzato dal TCP

Controllo di congestione assistito dalla rete
i router forniscono un feedback esplicito ai nodi terminali riguardante lo stato di congestione nella rete
misura della congestione nei router: lunghezza della coda dei buffer dei router
feedback diretto oppure aggiornando un campo del pacchetto che viaggia tra i nodi terminali

Controllo della congestione nel TCP

– **Segue l'approccio end-to-end**
nessuna informazione proveniente dalla rete

Soluzione adottata molto semplice da spiegare:
quando si verificano alti tassi di perdita di segmenti, e alte percentuali di ritrasmissione, vuol dire che c'è congestione
alloraTCP riduce il tasso di trasmissione (il numero di segmenti trasmessi per unità di tempo) in caso di congestione

In verità:
- determinare se la rete è intasato
- determinare come e quanto ridurre il tasso di trasmissione per ridurre la congestione

Sono cose molto difficili da fare…ci sono calcoli probabilistici, teoremi matematici relativi alla teoria delle code e del traffico….